Programación Avanzada
IIC2233 2026-2

Anuncios

15 de octubre de 2026



- Recuerden **marcar su salida con la TUC**, sino quedarán como ausentes.
- La próxima semana tendremos la **I2** en **horario de cátedra**.
  - Aprovechen el Ejercicio Propuesto 2 para estudiar.
  - La I2 evaluará hasta el contenido de esta semana: **Serialización y Networking 2**

Control de Entrada



`git add, git commit, git push`

Comentarios AC 6

- **Lean todo el enunciado** antes de ponerse a programar.
- **No utilicen los puertos:** 1234, 5000, 7860, 8888, 11434.
- Pueden hacer la **Parte 2 sin haber implementado el cifrado de la Parte 1**.
- Recuerden que esto es una **actividad individual. Mantengan el silencio.**
- **Import al inicio de servidor.py debe ser:**

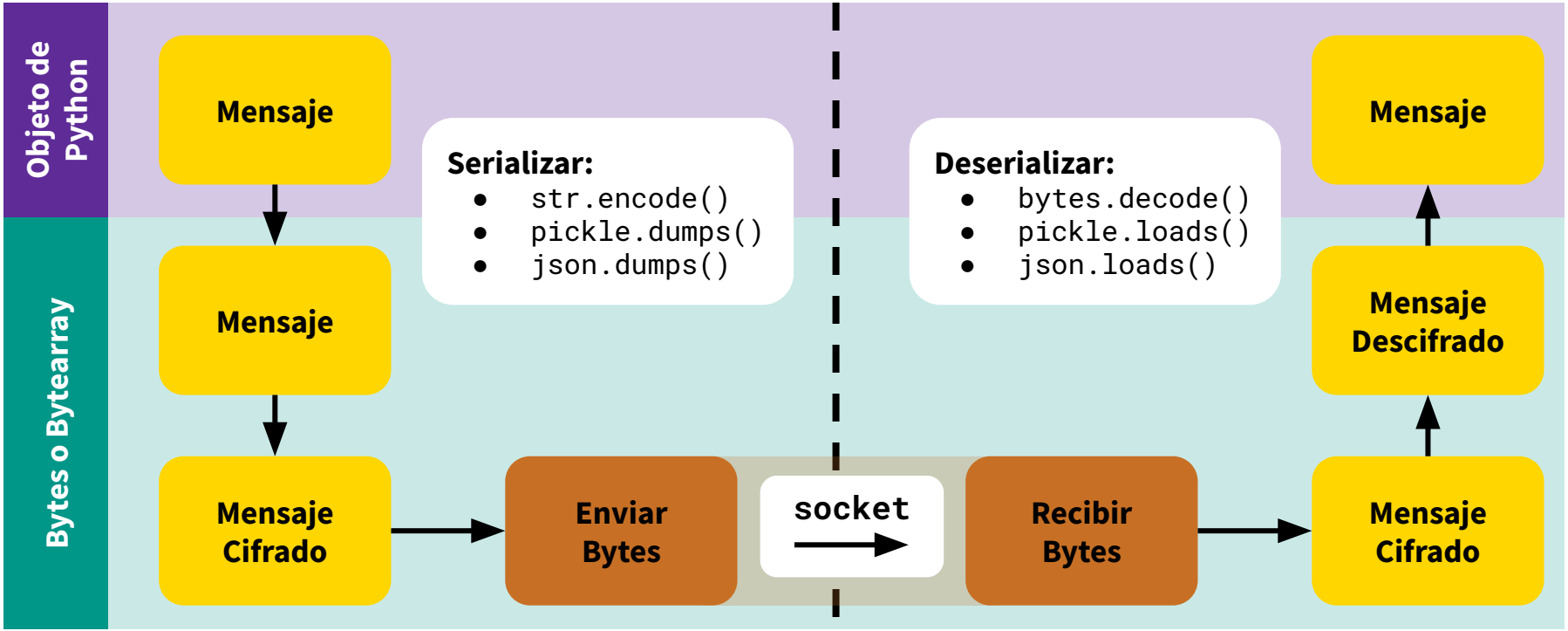
```
from cifrado import cifrar_bytes_mensaje, descifrar_bytes_mensaje
```

Cierre

Actividad 6: Serialización y Networking 2

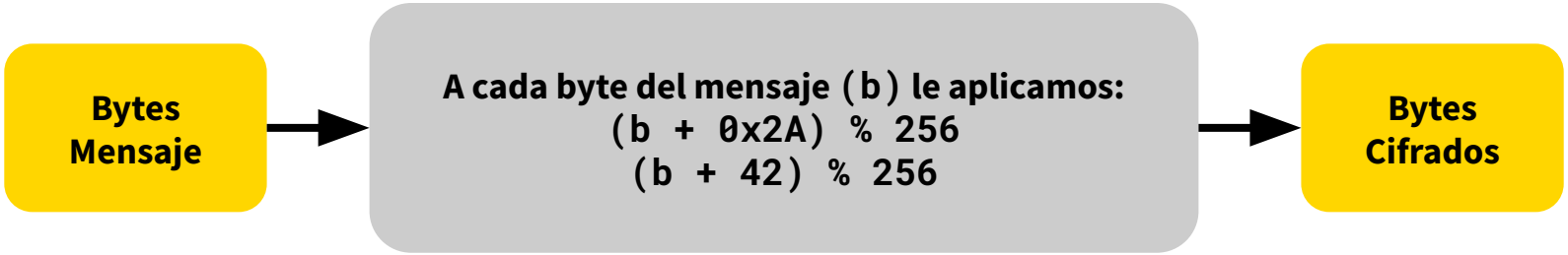


Flujo de mensajes



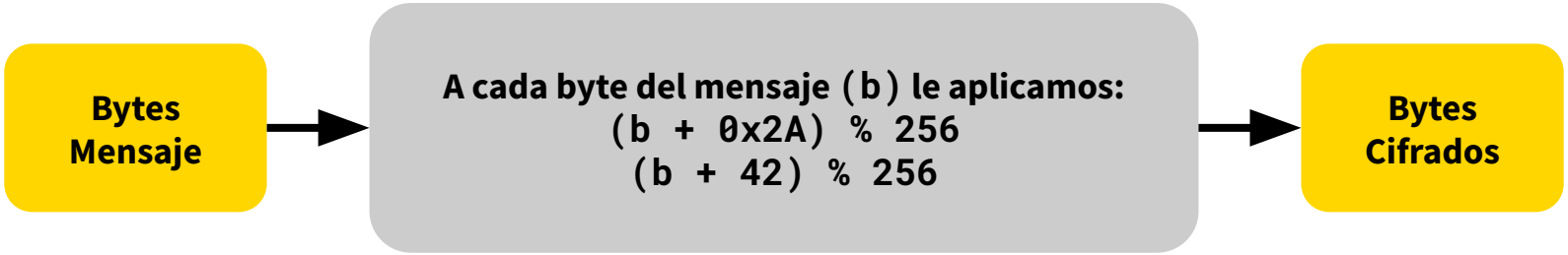
Cifrado y Descifrado de bytes

Cifrado

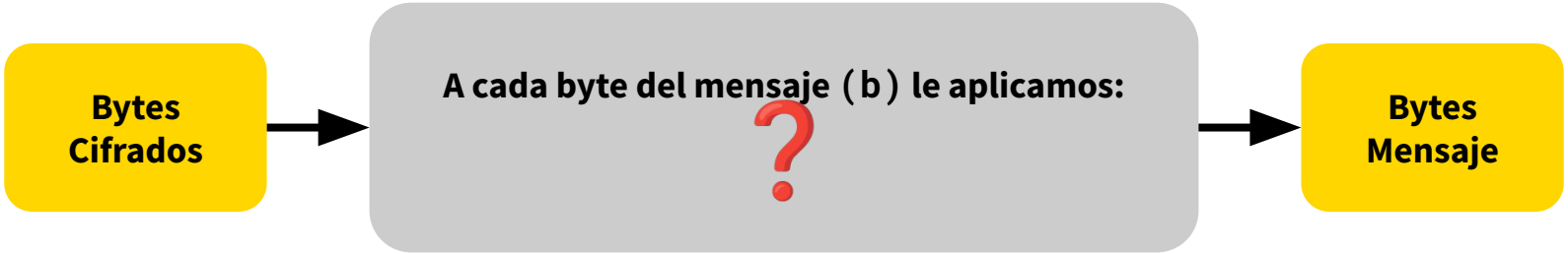


Cifrado y Descifrado de bytes

Cifrado

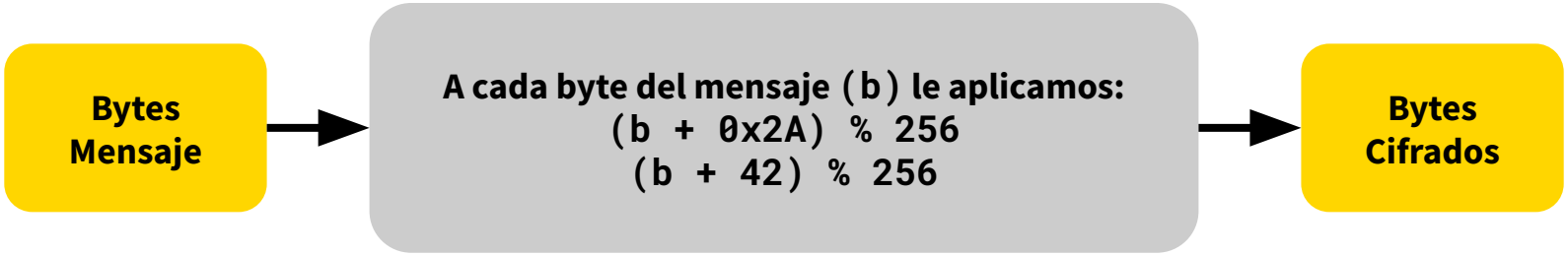


Descifrado

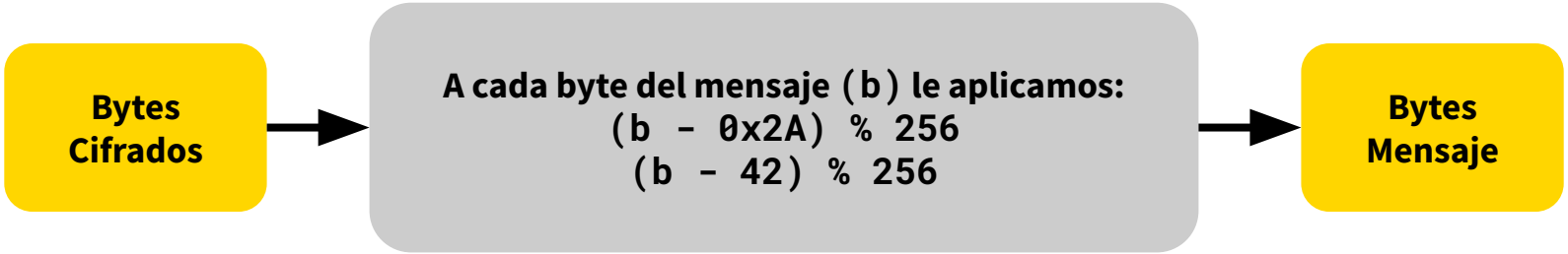


Cifrado y Descifrado de bytes

Cifrado



Descifrado



Responsabilidades Servidor vs Cliente

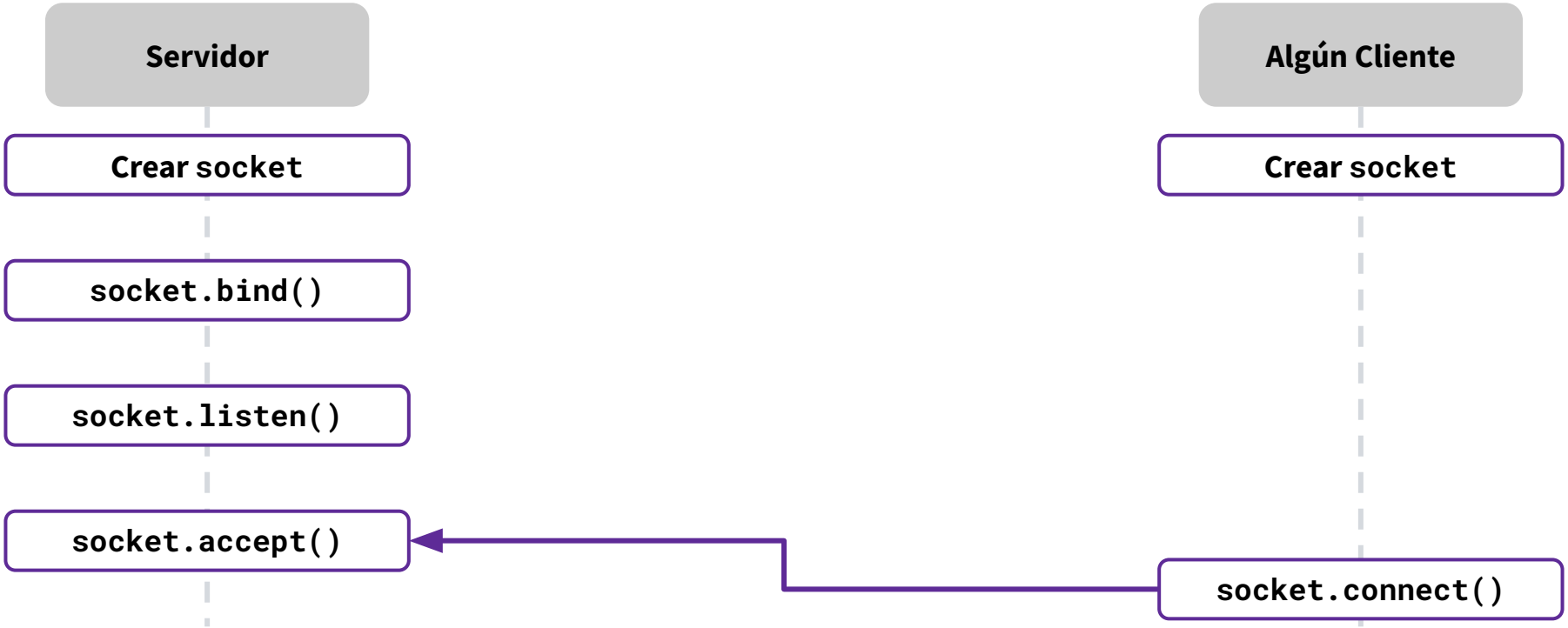
Servidor

- Recibir los 3 clientes.
- Administrar el juego.
  - Manejar las rondas de los jugadores.
  - Solicitar y recibir respuestas.
  - Validar cantidad aciertos.
  - Finalizar el juego.

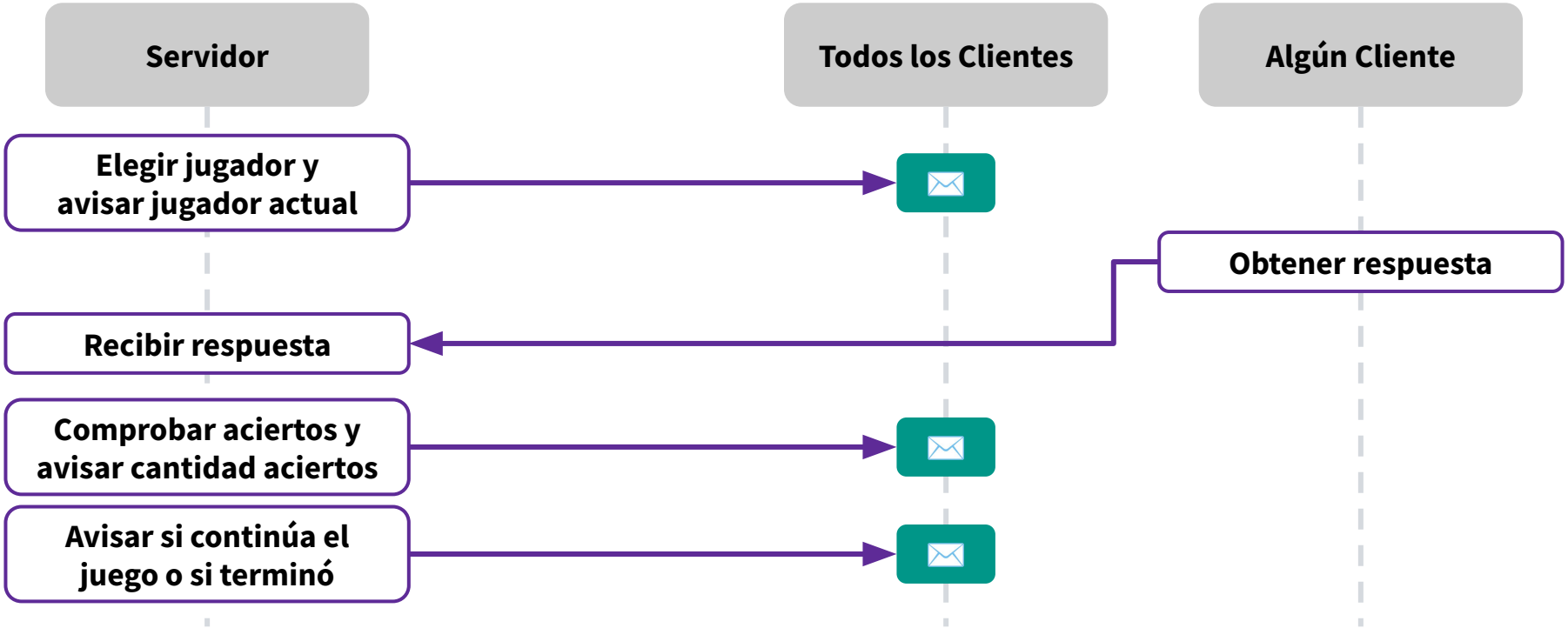
Cliente

- Mostrar estado del juego.
- Pedir respuesta al usuario.
- Mostrar cantidad aciertos.

Flujo del Programa



Flujo del Programa



Clase `Servidor`: métodos de *sockets*

```python
def aceptar_cliente(self) -> None:
    while len(self.clientes) < self.total_clientes:
        socket_cliente, direccion = self.socket_servidor.accept()
        self.clientes[direccion] = socket_cliente
        self.cola_clientes.append(direccion)
        print(f'[SERVIDOR] Cliente {direccion} conectado')
```

```python
def cerrar_sockets(self) -> None:
    print('[SERVIDOR] Cerrando sockets con clientes')
    for cliente, socket_cliente in self.clientes.items():
        print(f'[SERVIDOR] {cliente} conexión cerrada')
        socket_cliente.close()

    print('[SERVIDOR] Cerrando socket principal')
    self.socket_servidor.close()
```

```python
def recibir_mensaje(self, cliente) -> str:
    socket_cliente = self.clientes[cliente]
    bytes_mensaje = socket_cliente.recv(500)
    bytes_descifrados = descifrar_bytes_mensaje(bytes_mensaje)
    mensaje = bytes_descifrados.decode('UTF-8')
    return mensaje
```

```python
def enviar_mensaje_individual(self, mensaje: str, cliente) -> None:
    bytes_mensaje = mensaje.encode('UTF-8')
    bytes_cifrados = cifrar_bytes_mensaje(bytes_mensaje)
    socket_cliente = self.clientes[cliente]
    socket_cliente.sendall(bytes_cifrados)
```

```python
def enviar_mensaje_todos(self, mensaje: str) -> None:
    for cliente in self.clientes:
        self.enviar_mensaje_individual(mensaje, cliente)
```

Clase `Servidor`: métodos del juego

```python
def iniciar_juego(self) -> None:
    self.juego.desordenar_elementos()
    self.juego_activo = True

    print(f'[SERVIDOR] Juego iniciado. Respuesta correcta '
          f'{self.juego.elementos}\n')

    while self.juego_activo:
        self.jugar_turno()
```

```python
def jugar_turno(self) -> None:
    turno_actual = self.cola_clientes.popleft()

    print(f'[SERVIDOR] Turno de {turno_actual}')
    self.enviar_mensaje_todos(f'Turno de {turno_actual}')

    print(f'[SERVIDOR] Esperando respuesta de {turno_actual}')
    self.enviar_mensaje_individual('Es tu turno, envia tu respuesta', turno_actual)
    respuesta = self.recibir_mensaje(turno_actual)
    respuesta = [int(i) for i in respuesta]

    aciertos = self.juego.comprobar_elementos(respuesta)
    print(f'[SERVIDOR] {turno_actual} acertó {aciertos} de {self.juego.cantidad_elementos}')
    self.enviar_mensaje_todos(f'Acerto {aciertos} de {self.juego.cantidad_elementos}')

    if aciertos == self.juego.cantidad_elementos:
        print(f'[SERVIDOR] {turno_actual} resolvió el juego')
        self.enviar_mensaje_todos(f'Juego completado por {turno_actual}')
        self.juego_activo = False
    else:
        print(f'[SERVIDOR] El juego continua')
        self.enviar_mensaje_todos('Juego continua')
        self.cola_clientes.append(turno_actual)
```

Clase `Cliente`: métodos *socket* y juego

```python
def conectar_servidor(self) -> None:
    try:
        self.socket.connect((self.host, self.port))
        print(f'[CLIENTE ] Conectado en {self.socket.getsockname()}')
    except ConnectionError:
        direccion_probada = (self.host, self.port)
        print(f'[CLIENTE ] No fue posible conectarse con {direccion_probada}')
        self.socket.close()
        exit()


def recibir_mensaje(self) -> str:
    bytes_mensaje = self.socket.recv(500)
    bytes_descifrados = descifrar_bytes_mensaje(bytes_mensaje)
    mensaje = bytes_descifrados.decode('UTF-8')
    return mensaje


def enviar_mensaje(self, mensaje: str) -> None:
    bytes_mensaje = mensaje.encode('UTF-8')
    bytes_cifrados = cifrar_bytes_mensaje(bytes_mensaje)
    self.socket.sendall(bytes_cifrados)


def cerrar_socket(self):
    self.socket.close()
```

```python
def jugar_juego(self) -> None:
    while True:
        mensaje_servidor = self.recibir_mensaje()

        if not mensaje_servidor:
            self.cerrar_socket()
            return

        print('[SERVIDOR]', mensaje_servidor)

        if 'Es tu turno, envia tu respuesta' == mensaje_servidor:
            respuesta = input('[CLIENTE ] > ')
            self.enviar_mensaje(respuesta)

        if 'Juego completado por' in mensaje_servidor:
            print('[CLIENTE ] Juego terminado')
            self.cerrar_socket()
            return
```